

คำนำ

รายงานการออกแบบและพัฒนาโปรแกรมระบบร้านกาแฟด้วยภาษาไพทอน มีวัตถุประสงค์เพื่อพัฒนาโปรแกรมสำหรับการจัดการการขายเครื่องดื่มและเบเกอรี่ของร้านกาแฟแห่งหนึ่ง โครงการนี้จึงมุ่งเน้นที่จะนำเสนอเกี่ยวกับการพัฒนาโปรแกรมสำหรับการจัดการการขายเครื่องดื่มและเบเกอรี่ของร้านกาแฟแห่งหนึ่ง เพื่อผู้ที่สนใจสามารถนำโปรแกรมนี้ไปพัฒนาต่อยอดได้ในอนาคต

คณะผู้จัดทำจึงหวังว่า รายงานเล่มนี้จะเป็นประโยชน์กับผู้อ่านหรือนักเรียน นักศึกษาที่กำลังหาข้อมูลเรื่องนี้อยู่ หากมีข้อเสนอแนะหรือข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับไว้และขออภัยมา ณ ที่นี้ด้วย

คณะผู้จัดทำ

สารบัญ

คำนำ.....	ข
สารบัญ.....	ค
สารบัญรูปภาพ.....	ง
ความเป็นมาและความสำคัญ.....	1
วัตถุประสงค์.....	1
ประโยชน์ที่ได้รับ	1
ขอบเขตของโครงการ	2
ขั้นตอนและการดำเนินงาน.....	2
ผลการดำเนินงาน.....	21
สรุปผลการดำเนินงาน	27
บรรณานุกรม.....	28

สารบัญรูปภาพ

ภาพที่ 1 ไฟล์ menu.txt.....	2
ภาพที่ 2 ไฟล์ staff.txt.....	2
ภาพที่ 3 ไฟล์ sale.txt.....	3
ภาพที่ 4 หน้าเมนู.....	21
ภาพที่ 5 หน้าจอแสดงการเพิ่มเมนู.....	21
ภาพที่ 6 หน้าจอแสดงการแก้ไขเมนู.....	22
ภาพที่ 7 หน้าจอแสดงการลบข้อมูลของเมนู.....	22
ภาพที่ 8 หน้าจอแสดงการเพิ่มพนักงานขาย.....	22
ภาพที่ 9 หน้าจอแสดงการแก้ไขข้อมูลของพนักงานขาย.....	23
ภาพที่ 10 หน้าจอแสดงการลบข้อมูลของพนักงานขาย.....	23
ภาพที่ 11 หน้าจอแสดงการบันทึกการขาย.....	24
ภาพที่ 12 หน้าจอแสดงการบันทึกการขาย.....	24
ภาพที่ 13 หน้าจอแสดงการยกเลิกออเดอร์.....	25
ภาพที่ 14 หน้าจอแสดงการออกรายงานสรุปยอดขายประจำวัน.....	25
ภาพที่ 15 หน้าจอแสดงการออกรายงานสรุปยอดขายแยกตามพนักงาน.....	25
ภาพที่ 16 หน้าจอแสดงการคิดค่าคอมมิชชั่นของพนักงาน.....	26
ภาพที่ 17 หน้าจอแสดงออกจากโปรแกรม.....	26

ความเป็นมาและความสำคัญ

ในยุคที่ร้านค้าปลีกกลายเป็นสถานที่ยอดนิยมสำหรับการพบปะและทำงาน ความต้องการในการจัดการร้านให้มีประสิทธิภาพมากยิ่งขึ้นก็เพิ่มขึ้นตามไปด้วย เจ้าของร้านค้าปลีกว่าการบริหารจัดการในหลาย ๆ ด้าน เช่น การขายเครื่องดื่มและเบเกอรี่ การจัดการข้อมูลพนักงาน และการวิเคราะห์ยอดขาย เป็นเรื่องที่ซับซ้อนและต้องใช้เวลาอย่างมาก โดยกลุ่มของเราได้ทำการ ออกแบบและพัฒนาโปรแกรมระบบร้านค้าปลีกด้วย ภาษา Python

การจัดเก็บข้อมูลเครื่องดื่มและเบเกอรี่: โปรแกรมช่วยให้เจ้าของร้านสามารถบันทึกข้อมูลเกี่ยวกับเครื่องดื่มและเบเกอรี่ เช่น รหัสสินค้า ชื่อ ราคา โดยสามารถสร้างรายงานเพื่อดูข้อมูลเหล่านี้ได้อย่างง่ายดาย

การจัดเก็บข้อมูลพนักงานขาย: การบันทึกข้อมูลพนักงาน เช่น ชื่อ จะช่วยให้สามารถติดตามประสิทธิภาพการทำงานของแต่ละคน และสามารถสร้างรายงานเพื่อวิเคราะห์ผลการทำงานได้

การบันทึกข้อมูลการขาย: โปรแกรมสามารถบันทึกข้อมูลการขายเครื่องดื่มและเบเกอรี่ที่พนักงานขายได้ ทำให้สามารถติดตามยอดขายของแต่ละวันได้อย่างแม่นยำ

การออกรายงานยอดขาย: โปรแกรมสามารถสร้างรายงานสรุปยอดขายประจำวัน รายงานยอดขายตามพนักงานขาย และรายงานยอดขายตามข้อมูลเครื่องดื่มและเบเกอรี่ ซึ่งช่วยให้เจ้าของร้านสามารถวิเคราะห์แนวโน้มการขายและปรับกลยุทธ์ทางการตลาดได้อย่างเหมาะสม

การคำนวณค่า Commission: โปรแกรมสามารถคำนวณค่าคอมมิชชั่นของพนักงานขายได้อย่างอัตโนมัติ ทำให้การบริหารจัดการเงินเดือนและค่าตอบแทนเป็นไปอย่างราบรื่น

โดยโปรแกรมการออกแบบและพัฒนาโปรแกรมระบบร้านค้าปลีกด้วยภาษา Python มีความสำคัญในด้านการเพิ่มประสิทธิภาพการทำงานของร้านค้าปลีก โดยช่วยให้การจัดการข้อมูลเป็นไปอย่างมีระบบ สามารถบันทึกและวิเคราะห์ข้อมูลได้ง่ายขึ้น ซึ่งส่งผลให้เจ้าของร้านสามารถให้บริการลูกค้าได้ดียิ่งขึ้น และเป็นส่วนสำคัญในการสร้างความสำเร็จและการเติบโตของร้านค้าปลีกในตลาดที่มีการแข่งขันสูงในปัจจุบัน

วัตถุประสงค์

1. เพื่อฝึกพัฒนาโปรแกรมสำหรับภาษาไพทอน
2. เพื่อให้ผู้เรียนสามารถนำความรู้การเขียนโปรแกรมที่ได้รับไปประยุกต์ใช้ในธุรกิจต่างๆ หรือต่อยอดในการศึกษาระดับสูงต่อไป

ประโยชน์ที่ได้รับ

1. ได้รับโปรแกรมที่จัดเก็บข้อมูลพนักงานขาย เครื่องดื่มและเบเกอรี่ได้อย่างมีประสิทธิภาพ
2. ได้รับความแม่นยำและสามารถติดตามยอดขายของแต่ละวันได้อย่างมีประสิทธิภาพ
3. สามารถคำนวณค่าคอมมิชชั่นของพนักงานขายได้อย่างแม่นยำ

ขอบเขตของโครงการ

1. การออกแบบระบบโปรแกรม มีดังนี้
 - 1.1 โปรแกรมมีการทำงานด้วยเมนู
 - 1.2 โปรแกรมมีการจัดเก็บข้อมูลเครื่องดื่ม เบเกอรี่ และพนักงานขาย
 - 1.3 โปรแกรมมีการบันทึกการขายเครื่องดื่ม และเบเกอรี่
 - 1.4 โปรแกรมมีการออกรายงานสรุยอดขายประจำวัน
 - 1.5 โปรแกรมมีการออกรายงานสรุยอดขายแยกตามพนักงาน
 - 1.6 โปรแกรมมีการคำนวณหาค่าคอมมิชชั่นของพนักงานขาย

ขั้นตอนและการดำเนินงาน

1) การออกแบบโครงสร้างข้อมูลที่ใช้เก็บข้อมูล

โปรแกรมนี้ใช้ไฟล์ menu.txt และ staff.txt ในการเก็บข้อมูล โดยมีโครงสร้างไฟล์ดังนี้

menu.txt

- ใช้สำหรับเก็บรายการอาหาร โดยข้อมูลแต่ละรายการจะถูกเก็บในรูปแบบ รหัสเมนู, ชื่อเมนู, ราคา

```
101 | Green Tea | 30.0
102 | Cake | 60.0
```

ภาพที่ 1 ไฟล์ menu.txt

- โครงสร้างนี้ช่วยให้ข้อมูลแต่ละบรรทัดเป็นหนึ่งรายการในเมนู และสามารถแปลงเป็นข้อมูลที่เข้าใจง่ายในรูปแบบของรายการ Python

staff.txt

- ใช้สำหรับเก็บข้อมูลพนักงานในร้าน โดยข้อมูลแต่ละบรรทัดจะอยู่ในรูปแบบ รหัสพนักงาน, ชื่อพนักงาน

```
0001 | Ken
0002 | Jenny
|
```

ภาพที่ 2 ไฟล์ staff.txt

- โครงสร้างนี้ทำให้ระบบสามารถจัดการข้อมูลพนักงานได้สะดวกและมีข้อมูลที่เป็นระเบียบ
- sale.txt

- สำหรับการเก็บประวัติการขายของแต่ละออเดอร์
- แต่ละบรรทัดอาจประกอบด้วยข้อมูลเช่น รหัสเมนู, รหัสพนักงาน, จำนวน, ราคารวม

```
101 | 0001 | 2 | 60.0
102 | 0001 | 1 | 60.0
```

ภาพที่ 3 ไฟล์ sale.txt

2) แนวคิดการทำงานของโปรแกรม

แนวคิดของโปรแกรมนี้อคือการสร้างระบบที่สามารถจัดการข้อมูลร้านอาหารในหลายๆ ด้าน ซึ่งประกอบด้วย

การจัดการเมนู: ผู้ใช้สามารถเพิ่ม, ลบ หรือแก้ไขข้อมูลในเมนูได้ เช่น เพิ่มรายการอาหารใหม่, เปลี่ยนแปลงราคาของเมนูที่มีอยู่ หรือการลบเมนู

การจัดการพนักงาน: ระบบนี้รองรับการเพิ่มข้อมูลพนักงานใหม่, แก้ไขข้อมูลของพนักงานที่มีอยู่ และลบข้อมูลพนักงานหากจำเป็น

การบันทึกการขาย: ระบบสามารถบันทึกข้อมูลการขายในแต่ละวัน โดยจะบันทึกรายละเอียดของพนักงานที่ขาย รายการอาหารที่ขาย และยอดรวมของแต่ละการขาย

การยกเลิกคำสั่งซื้อ: มีฟังก์ชันที่ช่วยยกเลิกออเดอร์ที่บันทึกไปแล้ว เพื่อให้สะดวกในการแก้ไขข้อมูลเมื่อเกิดข้อผิดพลาด

การรายงานยอดขาย: ฟังก์ชันการรายงานยอดขายในแต่ละวันหรือยอดขายแยกตามพนักงาน ช่วยให้สามารถวิเคราะห์การทำงานของร้านได้ดียิ่งขึ้น

การคำนวณหาค่าคอมมิชชั่น: ฟังก์ชันการคำนวณค่าคอมมิชชั่นจะคำนวณให้พนักงานแต่ละคนตามยอดขายของพวกเขา โดยอัตราค่าคอมมิชชั่นจะขึ้นอยู่กับยอดขายรวมดังนี้

- ยอดขายต่ำกว่า 2000 บาท: อัตราค่าคอมมิชชั่น 2%
- ยอดขายตั้งแต่ 2000 ถึง 5000 บาท: อัตราค่าคอมมิชชั่น 3%
- ยอดขายมากกว่า 5000 บาท: อัตราค่าคอมมิชชั่น 4%

3) การทำงานของโปรแกรม

3.1 ฟังก์ชัน load_data

ฟังก์ชันนี้ทำหน้าที่ในการโหลดข้อมูลจากไฟล์ข้อความ (เช่น ไฟล์เมนูหรือไฟล์ข้อมูลพนักงาน) ซึ่งอยู่ในรูปแบบของไฟล์ .txt โดยมีรายละเอียดการทำงานดังนี้

```
def load_data(file_name):
    data = []
    try:
        if os.path.exists(file_name):
            with open(file_name, 'r', encoding='utf-8') as file:
                for line in file:
                    data.append(line.strip().split('|'))
        else:
            print(f"Warning: {file_name} does not exist.")
    except Exception as e:
        print(f"Error loading data from {file_name}: {e}")
    return data
```

1. ตรวจสอบการมีอยู่ของไฟล์

- ฟังก์ชันจะเช็คก่อนว่าไฟล์ที่ต้องการอ่านนั้นมีอยู่ในระบบหรือไม่ โดยใช้ `os.path.exists(file_name)`
- หากไฟล์มีอยู่ในระบบ จะเปิดไฟล์นั้นขึ้นมาเพื่ออ่านข้อมูล แต่หากไฟล์ไม่มีอยู่ จะขึ้นข้อความแจ้งเตือนว่าไฟล์ไม่พบ

2. เปิดไฟล์

- ใช้ `with open(file_name, 'r', encoding='utf-8')` เพื่อเปิดไฟล์ในโหมดการอ่าน ('r') และกำหนด encoding เป็น 'utf-8' เพื่อรองรับการอ่านข้อมูลภาษาไทย
- `with` ใช้เพื่อให้โปรแกรมจัดการปิดไฟล์ให้โดยอัตโนมัติเมื่อการทำงานเสร็จสิ้น

3. อ่านข้อมูลที่ละบรรทัด

- วนลูปอ่านแต่ละบรรทัดในไฟล์ โดยใช้ `for line in file`
- ใช้ `line.strip().split('|')` เพื่อลบช่องว่างส่วนเกินจากต้นและท้ายของบรรทัด (`strip()`) แล้วแบ่งข้อมูลออกเป็นรายการย่อยตามสัญลักษณ์ `|`
- ผลลัพธ์จะถูกเพิ่มเข้าไปในลิสต์ `data` ในแต่ละบรรทัด

4. จัดการข้อผิดพลาด

- หากเกิดข้อผิดพลาดในการเปิดหรืออ่านไฟล์ จะทำการแสดงข้อความแจ้งเตือนผ่าน `print(f"Error loading data from {file_name}: {e}")`

5. คืนค่าผลลัพธ์

- เมื่ออ่านข้อมูลทั้งหมดเสร็จสิ้น ฟังก์ชันจะคืนค่า `data` ซึ่งเป็นลิสต์ของข้อมูลที่อ่านได้จากไฟล์นั้น ๆ

3.2 ฟังก์ชัน `save_data`

ฟังก์ชันนี้ทำหน้าที่ในการบันทึกข้อมูลลงไฟล์ โดยจะเขียนข้อมูลใหม่ทับข้อมูลเดิมทั้งหมดในไฟล์

```
def save_data(data, file_name):
    try:
        with open(file_name, 'w', encoding='utf-8') as file:
            for item in data:
                file.write(' '.join(map(str, item)) + '\n')
    except Exception as e:
        print(f"Error saving data to {file_name}: {e}")
```

1. เปิดไฟล์ในโหมดเขียน ('w')

- ใช้ `with open(file_name, 'w', encoding='utf-8')` เพื่อเปิดไฟล์ในโหมดการเขียน
- `encoding='utf-8'` ช่วยรองรับการบันทึกข้อมูลที่มีภาษาไทย

2. เขียนข้อมูลลงไฟล์

- วนลูปเพื่อเขียนข้อมูลแต่ละรายการจากลิสต์ `data` ไปที่ไฟล์
- ใช้ `' '.join(map(str, item)) + '\n'` ในการรวมข้อมูลแต่ละรายการใน `item` โดยใช้ `|` เป็นตัวคั่น และ `\n` เพื่อขึ้นบรรทัดใหม่

3. จัดการข้อผิดพลาด

- หากเกิดข้อผิดพลาดในการบันทึกข้อมูล จะทำการแสดงข้อความแจ้งเตือนผ่าน `print(f"Error saving data to {file_name}: {e}")`

3.3 ฟังก์ชัน `add_menu`

ฟังก์ชันนี้ใช้สำหรับการจัดการเมนู ซึ่งรวมถึงการเพิ่ม แก้ไข และลบเมนู รวมถึงการแสดงผลเมนูทั้งหมดในรูปแบบตาราง โดยมีการทำงานดังนี้

```
def add_menu(menu_file='menu.txt'):
    try:
        menu_items = load_data(menu_file)
        menu_dict = {item[0]: [item[1], float(item[2])]} for item in menu_items
    while True:
```

```

        action = input("Do you want to add, edit, or delete an
item? (add/edit/delete): ").lower()

        if action == "edit":
            item_id = input("Enter item ID to edit: ")

            if item_id in menu_dict:
                print(f"Editing Item ID {item_id}:
{menu_dict[item_id][0]} - Current Price: {menu_dict[item_id][1]}")
                name = input(f"Enter new name for {menu_dict[item_id][0]} (leave
blank to keep current name): ")
                price_input = input(f"Enter new price for {menu_dict[item_id][0]}
(current price: {menu_dict[item_id][1]}): ")

                if name:
                    menu_dict[item_id][0] = name
                if price_input:
                    menu_dict[item_id][1] = float(price_input)

                print(f"Item ID {item_id} has been updated.")
            else:
                print(f"Item ID {item_id} does not exist.")

        elif action == "add":
            item_id = input("Enter item ID: ")

            if item_id not in menu_dict:
                name = input("Enter item name: ")
                price = float(input("Enter price: "))
                menu_dict[item_id] = [name, price]
                print(f"{name} has been added to the menu.")
            else:
                print(f"Item ID {item_id} already exists. You can
edit it instead.")

        elif action == "delete":
            item_id = input("Enter item ID to delete: ")

            if item_id in menu_dict:
                del menu_dict[item_id]
                print(f"Item ID {item_id} has been deleted from
the menu.")
            else:
                print(f"Item ID {item_id} does not exist.")
            else:
                print("Invalid action. Please choose 'add',
'edit', or 'delete'.")

        save_data([[item_id, menu_dict[item_id][0],
menu_dict[item_id][1]] for item_id in menu_dict], menu_file)

```

```

        print('-'*44)
        print("Current Menu".center(42))
        print('-'*44)
        print(f"|{'Item ID':<5}| {' Menu Item ':15}| {' Price (Baht) ':15}|")
        print('-' * 44)
        for item_id, (name, price) in menu_dict.items():
            print(f"|{item_id:<7}|{name:<17}| {price:>15.2f}|")
        print('-'*44)
        print()

        if input("Add, edit, or delete another item? (y/n): ").lower() != 'y':
            break
    except ValueError as ve:
        print(f"Invalid input: {ve}")
    except Exception as e:
        print(f"Error adding, editing, or deleting menu: {e}")

```

1. โหลดเมนูจากไฟล์

- เรียก load_data(menu_file) เพื่อโหลดข้อมูลเมนูที่บันทึกไว้ในไฟล์ menu.txt
- ข้อมูลที่ได้จะถูกแปลงเป็น menu_dict ซึ่งเป็น dictionary ที่ใช้ item_id เป็น key และมีค่าเป็นลิสต์ของชื่อเมนูและราคา

2. วนลูปเพื่อทำการเพิ่ม แก้ไข หรือ ลบเมนู

- โปรแกรมจะให้ผู้ใช้เลือก add, edit, หรือ delete ตามที่ต้องการ และรับค่าข้อมูลตามที่ต้องการ
 - add: เพิ่มเมนูใหม่ โดยรับ รหัสเมนู, ชื่อเมนู, และราคา
 - edit: แก้ไขข้อมูลของเมนูที่มีอยู่ โดยสามารถอัปเดตชื่อและราคาของเมนู
 - delete: ลบเมนูที่มีอยู่ตามรหัสเมนู

3. บันทึกข้อมูลเมนูกลับไปไฟล์

- เรียก save_data เพื่อบันทึกข้อมูลเมนูที่อัปเดตแล้วกลับไปยังไฟล์ menu.txt

4. แสดงเมนูในรูปแบบตาราง

- ใช้การจัดการข้อความเพื่อให้แสดงข้อมูลเมนูในรูปแบบตาราง โดยแสดง รหัสเมนู, ชื่อเมนู, และราคาในแต่ละบรรทัด
- ทำการจัดรูปแบบเพื่อให้อ่านง่าย

5. ถามผู้ใช่ว่าจะทำการเพิ่ม แก้ไข หรือลบเมนูต่อหรือไม่

- หากผู้ใช้ตอบ y จะกลับไปเริ่มต้นใหม่, แต่ถ้า n จะออกจากฟังก์ชัน

3.4 ฟังก์ชัน add_staff

ฟังก์ชันนี้ใช้สำหรับการจัดการข้อมูลพนักงาน โดยมีโครงสร้างการทำงานคล้ายกับ

add_menu ดังนี้

```
def add_staff(staff_file='staff.txt'):
    try:
        staff_data = load_data(staff_file)
        staff_dict = {staff[0]: staff[1] for staff in staff_data}

        while True:
            action = input("Do you want to add, edit, or delete a
staff member? (add/edit/delete): ").lower()

            if action == "edit":
                staff_id = input("Enter staff ID to edit: ")

                if staff_id in staff_dict:
                    print(f"Editing Staff ID {staff_id}:
{staff_dict[staff_id]}")
                    name = input(f"Enter new name for
{staff_dict[staff_id]} (leave blank to keep current name): ")

                    if name:
                        staff_dict[staff_id] = name
                        print(f"Staff ID {staff_id} has been updated.")
                    else:
                        print(f"Staff ID {staff_id} does not exist.")

            elif action == "add":
                staff_id = input("Enter staff ID: ")

                if staff_id not in staff_dict:
                    name = input("Enter staff name: ")
                    staff_dict[staff_id] = name
                    print(f"{name} has been added to the staff
list.")
                else:
                    print(f"Staff ID {staff_id} already exists. You
can edit it instead.")

            elif action == "delete":
                staff_id = input("Enter staff ID to delete: ")

                if staff_id in staff_dict:
                    del staff_dict[staff_id]
                    print(f"Staff ID {staff_id} has been deleted from
the staff list.")
                else:
                    print(f"Staff ID {staff_id} does not exist.")
```

```

        else:
            print("Invalid action. Please choose 'add', 'edit',
or 'delete'.")

            save_data([[staff_id, staff_dict[staff_id]] for staff_id
in staff_dict], staff_file)

            print('-'* 30)
            print("Current Staff".center(28))
            print('-'* 30)
            print(f"|{'Staff ID':<13}|{'Staff Name':<14}|")
            print('-' * 30)
            for staff_id, name in staff_dict.items():
                print(f"|{staff_id:<13}|{name:<14}|")
            print('-'*30)
            print()

            if input("Add, edit, or delete another staff member?
(y/n): ").lower() != 'y':
                break
    except Exception as e:
        print(f"Error adding, editing, or deleting staff: {e}")

```

1. โหลดข้อมูลพนักงาน

- เรียก load_data(staff_file) เพื่อนำข้อมูลพนักงานทั้งหมดเข้ามา
- ข้อมูลถูกจัดเก็บเป็น dictionary ที่มีรหัสพนักงานเป็น key และชื่อพนักงานเป็นค่า

2. วนลูปเพื่อทำการเพิ่ม แก้ไข หรือ ลบข้อมูลพนักงาน

- รับคำสั่งจากผู้ใช้ (add, edit, หรือ delete) เพื่อจัดการข้อมูลตามที่ต้องการ
 - add: เพิ่มพนักงานใหม่
 - edit: แก้ไขชื่อพนักงานที่มีอยู่
 - delete: ลบข้อมูลพนักงานตาม รหัสพนักงาน

3. บันทึกข้อมูลพนักงานกลับไปไฟล์

- เรียก save_data เพื่อบันทึกข้อมูลพนักงานที่อัปเดตแล้วกลับไปไฟล์ staff.txt

4. แสดงข้อมูลพนักงานในรูปแบบตาราง

- จัดข้อความให้แสดงรหัสพนักงาน และชื่อพนักงานในรูปแบบตารางที่อ่านง่าย

5. ตรวจสอบว่าจะทำการเพิ่ม แก้ไข หรือลบข้อมูลพนักงานอีกหรือไม่

- ถ้าผู้ใช้ตอบ y จะกลับไปทำต่อ, ถ้าตอบ n จะออกจากฟังก์ชัน

3.5 ฟังก์ชัน add_sale

ฟังก์ชันนี้ทำหน้าที่บันทึกการขายสินค้าและแสดงใบเสร็จ โดยการทำงานมีขั้นตอนดังนี้

```

def add_sale(menu_file='menu.txt', staff_file='staff.txt',
sales_file='sales.txt'):
    try:
        menu_items = load_data(menu_file)
        staff = load_data(staff_file)

        menu_dict = {item[0]: {'name': item[1], 'price': float(item[2])}}
    for item in menu_items}
        staff_dict = {s[0]: s[1] for s in staff}

        staff_id = input("Enter staff ID: ")
        if staff_id not in staff_dict:
            print("Invalid staff ID.")
            return
        current_sale = [ ]
        total_sale_amount = 0
        while True:
            item_id = input("Enter item ID: ")
            if item_id not in menu_dict:
                print("Invalid item ID. Please try again.")
                continue

            quantity = int(input("Enter quantity: "))
            item_total = menu_dict[item_id]['price'] * quantity
            total_sale_amount += item_total
            current_sale.append([item_id, menu_dict[item_id]['name'],
quantity, menu_dict[item_id]['price'], item_total])

            add_more = input("Do you want to add another item? (y/n):
").lower()
            if add_more != 'y':
                break

        sales = load_data(sales_file)
        for item in current_sale:
            sales.append([item[0], staff_id, item[2], item[4]]) # item_id,
staff_id, quantity, total
        save_data(sales, sales_file)

        print("=" * 50)
        print("Receipt".center(55))
        print("=" * 50)
        print(f"|{'Item':<20}|{'Qty':>5}|{'Price':>10}|{'Total':>10}|")
        print("-" * 50)
        for item in current_sale:
            print(f"|{item[1]:<20}|{item[2]:>5}|{item[3]:>10.2f}|{item[4]:>
10.2f}|")
        print("-" * 50)
        print(f"|{'Total':<35}|{total_sale_amount:>8.2f} Baht|")
        print("=" * 50)
        print(f"Sales recorded by staff: {staff_dict[staff_id]}")
        print("Thank you for your purchase!\n")

    except ValueError as ve:
        print(f"Invalid input: {ve}")
    except Exception as e:
        print(f"Error adding sale: {e}")

```

1. โหลดข้อมูลเมนูและพนักงาน

- ใช้ load_data โหลดข้อมูลจากไฟล์ menu_file และ staff_file แล้วจัดเก็บเป็น dictionary เพื่อง่ายต่อการอ้างอิง

2. รับข้อมูลการขาย

- รับ staff_id จากผู้ใช้เพื่อระบุพนักงานที่ทำการขาย ตรวจสอบว่ามี staff_id นั้นในระบบ
- วนลูปเพื่อรับ item_id และจำนวน (quantity) ของสินค้าที่ขาย จากนั้นคำนวณยอดรวมสำหรับสินค้านั้น

3. บันทึกการขายในไฟล์

- โหลดข้อมูลการขายเก่าจาก sales_file แล้วเพิ่มการขายใหม่เข้าไป
- ใช้ save_data เพื่อบันทึกการขายใหม่ในไฟล์ sales.txt

4. แสดงใบเสร็จในรูปแบบตาราง

- แสดงรายการสินค้า, จำนวน, ราคา และยอดรวม

3.4 ฟังก์ชัน cancel_order

ฟังก์ชันนี้ใช้สำหรับยกเลิกรายการขาย โดยมีรายละเอียดการทำงานดังนี้

```
def cancel_order(sales_file='sales.txt'):
    try:
        sales = load_data(sales_file)
        print("=" * 50)
        print("Current Orders".center(50))
        print("=" * 50)
        print(f"|{'Order No.':<9}|{'Item ID':<9}|{'Staff ID':<9}|{'Qty':<5}|{'Total (Baht)':>12}|")
        print("-" * 50)
        for index, sale in enumerate(sales, start=1):
            print(f"|{index:<9}|{sale[0]:<9}|{sale[1]:<9}|{sale[2]:>5}|{float(sale[3]):>12.2f}|")
            print("-" * 50)

        order_no = int(input("Enter the Order No. to cancel: "))
        if 1 <= order_no <= len(sales):
            canceled_order = sales.pop(order_no - 1)
            save_data(sales, sales_file)
            print(f"Order for Item ID {canceled_order[0]} by Staff ID {canceled_order[1]} has been canceled.")
        else:
            print("Invalid Order No.")
    except ValueError:
        print("Invalid input. Please enter a valid order number.")
    except Exception as e:
        print(f"Error canceling order: {e}")
```

1. โหลดข้อมูลการขาย

- ฟังก์ชันเริ่มด้วยการโหลดข้อมูลการขายจากไฟล์ sales.txt โดยเรียกใช้ฟังก์ชัน

load_data(sales_file) และเก็บข้อมูลที่ได้ในตัวแปร sales_data

- ข้อมูลที่ได้จะเป็นรายการการขายทั้งหมด ซึ่งมีข้อมูล sale_id, date, staff_id, รายการสินค้า, จำนวนสินค้า, และยอดรวมของการขาย

2. กรองข้อมูลการขายตามพนักงาน

- ฟังก์ชันจะสร้างรายการชื่อ staff_sales เพื่อเก็บข้อมูลการขายที่เฉพาะเจาะจงกับ staff_id ที่ได้รับมา
- โดยจะวนลูปผ่านข้อมูลใน sales_data แล้วตรวจสอบว่า staff_id ของแต่ละรายการตรงกับ staff_id ที่ระบุหรือไม่
- หากตรงกัน ข้อมูลการขายนั้นจะถูกเพิ่มเข้าไปใน staff_sales เพื่อสร้างเป็นรายงานเฉพาะพนักงานคนนั้น

3. แสดงรายงานการขาย

- ฟังก์ชันจะแสดงข้อมูลใน staff_sales ในรูปแบบตารางที่แสดง sale_id, date, รายการสินค้า, จำนวนสินค้า, และยอดขายรวม
- ตารางนี้ช่วยให้ผู้ใช้สามารถดูรายการขายทั้งหมดที่พนักงานคนนั้นทำได้ในแต่ละวัน

4. จัดการกรณีที่ไม่มีการขาย

- หาก staff_sales ว่างเปล่า นั่นหมายความว่าพนักงานคนนั้นไม่มีการขายในระบบ
- ในกรณีนี้ ฟังก์ชันจะแสดงข้อความแจ้งว่าไม่มีรายการขายสำหรับรหัสพนักงานที่ระบุ

3.5 ฟังก์ชัน generate_daily_sales_report

ฟังก์ชันนี้ใช้ในการสร้างรายงานการขายประจำวัน โดยมีรายละเอียดการทำงานดังนี้

```
def generate_daily_sales_report(sales_file='sales.txt',
                               menu_file='menu.txt'):
    try:
        sales_data = load_data(sales_file)
        menu_data = load_data(menu_file)

        menu_dict = {item[0]: item[1] for item in menu_data}

        sales_summary = {}
        total_sales = 0.0

        for sale in sales_data:
            item_id = sale[0]
            quantity = int(sale[2])
            total = float(sale[3])

            if item_id in sales_summary:
```

```

        sales_summary[item_id]['quantity'] += quantity
        sales_summary[item_id]['total'] += total
    else:
        sales_summary[item_id] = {
            'name': menu_dict.get(item_id, "Unknown Item"),
            'quantity': quantity,
            'total': total
        }

    total_sales += total

    print('-'*50)
    print("Daily Sales Report".center(50))
    print('-'*50)
    print(f"|{'Menu Item':<15}|{'Quantity Sold':>15}|{'Total (Baht)':>15}|")
    print('-' * 50)
    for item_id, summary in sales_summary.items():
        print(f"|{summary['name']:<15}|{summary['quantity']:15}|{summary['total']:>15.2f}|")
    print('-' * 50)
    print(f"|{'Total Sales':<15}|{'':>15}|{total_sales:>15.2f}|")
    print('-'*50)
except Exception as e:
    print(f"Error generating daily sales report: {e}")

```

1. โหลดข้อมูลการขาย

- ฟังก์ชันเริ่มต้นด้วยการเรียกใช้ load_data(sales_file) เพื่อโหลดข้อมูลการขายทั้งหมดจากไฟล์ sales.txt และเก็บไว้ในตัวแปร sales_data
- ข้อมูลที่ได้จะประกอบด้วยรายการการขายทั้งหมดในระบบ

2. การเพิ่มข้อมูลการขายเข้าใน sales_summary

- sales_summary เป็น Dictionary ที่ใช้เก็บข้อมูลยอดขายของแต่ละเมนูในรูปแบบที่มี item_id เป็น key โดยเก็บข้อมูล name (ชื่อเมนู), quantity (จำนวนที่ขายได้), และ total (ยอดขายรวม) สำหรับแต่ละเมนู
- ตัวแปร total_sales ใช้เก็บยอดขายรวมทั้งหมดของทุกเมนู
- ฟังก์ชันตรวจสอบว่า item_id ของสินค้านั้นมีอยู่ใน sales_summary แล้วหรือไม่
- หากมีอยู่แล้ว ฟังก์ชันจะเพิ่ม quantity และ total ของรายการการขายนี้เข้าไปในค่าที่มีอยู่
- หากยังไม่มี ฟังก์ชันจะสร้าง entry ใหม่ใน sales_summary โดยดึงชื่อเมนูจาก menu_dict ตาม item_id และกำหนด quantity และ total สำหรับเมนูนี้
- หากไม่มี item_id ใน menu_dict จะกำหนดชื่อเมนูเป็น "Unknown Item" แทน

3. คำนวณยอดขายรวมของวัน

- ในแต่ละรอบของการวนลูป ฟังก์ชันจะเพิ่มยอดขาย (total) ของรายการปัจจุบันเข้าไปในตัวแปร total_sales
- total_sales นี้จะเก็บยอดขายรวมของทุกรายการที่ขายได้ทั้งหมด5. จัดการกรณีที่ไม่มีการขายในวันนั้น

4. แสดงรายงานการขายประจำวัน

- ฟังก์ชันจะแสดงรายงานโดยใช้การพิมพ์ข้อความในรูปแบบตาราง โดยแสดงหัวข้อย่อยรายงานเป็น "Daily Sales Report"
- ใช้ f-string ในการจัดข้อความให้ตรงตำแหน่งโดยแบ่งเป็น Menu Item, Quantity Sold, และ Total (Baht) ซึ่งเป็นคอลัมน์แสดงชื่อเมนู จำนวนที่ขายได้ และยอดขายรวม
- ฟังก์ชันจะวนลูปผ่าน sales_summary เพื่อพิมพ์รายละเอียดการขายของแต่ละเมนูในตาราง
- นอกจากนี้ ฟังก์ชันจะแสดงยอดขายรวมทั้งหมด (total_sales) ในแถวสุดท้ายของตาราง เพื่อแสดงยอดรวมของการขายทั้งหมด

หากเกิดข้อผิดพลาดระหว่างการประมวลผล เช่น การอ่านไฟล์ผิดพลาด ฟังก์ชันจะจับข้อผิดพลาดและแสดงข้อความ "Error generating daily sales report" พร้อมรายละเอียดของข้อผิดพลาด (e)

3.6 ฟังก์ชัน generate_sales_report_by_staff

ฟังก์ชันนี้ใช้ในการสร้างรายงานการขายของพนักงานแต่ละคนตาม staff_id ที่ระบุ ซึ่งมีการทำงานดังนี้

```
def generate_sales_report_by_staff(sales_file='sales.txt',
    staff_file='staff.txt'):
    try:
        sales = load_data(sales_file)
        staff = load_data(staff_file)
        staff_dict = {s[0]: s[1] for s in staff}

        report = {}
        for sale in sales:
            staff_id = sale[1]
            total = float(sale[3])
            report[staff_id] = report.get(staff_id, 0) + total

        print('-'*35)
        print("Sales Report by Staff".center(35))
        print('-'*35)
        print(f"|{' Staff Name ':<15}|{'Total (Baht) ':>17}|")
        print('-' * 35)
        for staff_id, total in report.items():
```

```

        print(f"|{staff_dict.get(staff_id, 'Unknown
Staff'):<15}|{total:>17.2f}|")
        print('- '*35)
        print()
    except Exception as e:
        print(f"Error generating sales report by staff: {e}")

```

1. โหลดข้อมูลการขาย

- ฟังก์ชันเริ่มด้วยการโหลดข้อมูลการขายจากไฟล์ sales.txt โดยเรียกใช้ฟังก์ชัน `load_data(sales_file)` และเก็บข้อมูลที่ได้อินตัวแปร `sales_data`
- ข้อมูลที่ได้จะเป็นรายการการขายทั้งหมด ซึ่งมีข้อมูล `sale_id`, `date`, `staff_id`, รายการสินค้า, จำนวนสินค้า, และยอดรวมของการขาย

2. กรองข้อมูลการขายตามพนักงาน

- ฟังก์ชันจะสร้างรายการชื่อ `staff_sales` เพื่อเก็บข้อมูลการขายที่เฉพาะเจาะจงกับ `staff_id` ที่ได้รับมา
- โดยจะวนลูปผ่านข้อมูลใน `sales_data` แล้วตรวจสอบว่า `staff_id` ของแต่ละรายการตรงกับ `staff_id` ที่ระบุหรือไม่
- หากตรงกัน ข้อมูลการขายนั้นจะถูกเพิ่มเข้าไปใน `staff_sales` เพื่อสร้างเป็นรายงานเฉพาะพนักงานคนนั้น

3. แสดงรายงานการขาย

- ฟังก์ชันจะแสดงข้อมูลใน `staff_sales` ในรูปแบบตารางที่แสดง `sale_id`, `date`, รายการสินค้า, จำนวนสินค้า, และยอดขายรวม
- ตารางนี้ช่วยให้ผู้ใช้สามารถดูรายการขายทั้งหมดที่พนักงานคนนั้นทำได้ในแต่ละวัน

4. จัดการกรณีที่ไม่มีการขาย

- หาก `staff_sales` ว่างเปล่า นั่นหมายความว่าพนักงานคนนั้นไม่มีการขายในระบบ
- ในกรณีนี้ ฟังก์ชันจะแสดงข้อความแจ้งว่าไม่มีรายการขายสำหรับ `staff_id` ที่ระบุ หากเกิดข้อผิดพลาดระหว่างการประมวลผล เช่น การอ่านไฟล์ผิดพลาด ฟังก์ชันจะจับข้อผิดพลาดและแสดงข้อความ "Error generating sales report by staff" พร้อมรายละเอียดของข้อผิดพลาด (e)

3.7 ฟังก์ชัน `calculate_commission_rate`

ฟังก์ชันนี้จะถูกเรียกใช้ภายใน `calculate_commissions` เพื่อคำนวณอัตราค่าคอมมิชชั่นที่แตกต่างกันตามระดับของยอดขายรวม (`total_sales`)

```
def calculate_commission_rate(total_sales):
    if total_sales < 2000:
        return 0.02 # 2%
    elif 2000 <= total_sales <= 5000:
        return 0.03 # 3%
    else:
        return 0.04 # 4%
```

1. คำนวณอัตราคอมมิชชั่นตามยอดขายรวม

ฟังก์ชันใช้ if-elif-else เพื่อกำหนดอัตราคอมมิชชั่นตามเกณฑ์ต่างๆ ดังนี้

- หากยอดขายรวม (total_sales) มากกว่า 100,000 บาท อัตราคอมมิชชั่นจะถูกกำหนดเป็น 10% หรือ 0.10
- หากยอดขายรวมมากกว่า 50,000 บาท แต่ไม่เกิน 100,000 บาท อัตราคอมมิชชั่นจะเป็น 7% หรือ 0.07
- หากยอดขายรวมมากกว่า 10,000 บาท แต่ไม่เกิน 50,000 บาท อัตราคอมมิชชั่นจะเป็น 5% หรือ 0.05
- หากยอดขายรวมต่ำกว่า 10,000 บาท อัตราคอมมิชชั่นจะเป็น 3% หรือ 0.03

2. ส่งค่ากลับ (Return)

- หลังจากคำนวณอัตราคอมมิชชั่นตามเงื่อนไขที่ระบุ ฟังก์ชันจะส่งค่าของอัตราคอมมิชชั่น (commission_rate) กลับออกมา
- ค่านี้สามารถนำไปใช้ในการคำนวณคอมมิชชั่นของพนักงานแต่ละคนตามยอดขายรวมที่พวกเขาทำได้

3.8 ฟังก์ชัน calculate_commissions

ฟังก์ชันนี้มีขั้นตอนการทำงานตามลำดับดังนี้

```
def calculate_commissions(sales_file='sales.txt',
    staff_file='staff.txt'):
    try:
        sales = load_data(sales_file)
        staff = load_data(staff_file)
        staff_dict = {s[0]: s[1] for s in staff}

        sales_by_staff = {}
        for sale in sales:
            staff_id = sale[1]
            total = float(sale[3])
            sales_by_staff[staff_id] = sales_by_staff.get(staff_id,
0) + total

        commissions = {}
        for staff_id, total_sales in sales_by_staff.items():
            commission_rate = calculate_commission_rate(total_sales)
```

```

        commissions[staff_id] = total_sales * commission_rate

    print('-'*60)
    print("Staff Commissions".center(58))
    print('-'*60)
    print(f"{'|Staff Name':<18}| {'Total Sales (Baht)':>18}| {'Commission (Baht)':>18}|")
    print('-' * 60)
    for staff_id, commission in commissions.items():
        print(f"|{staff_dict.get(staff_id, 'Unknown Staff'):<17}| {sales_by_staff[staff_id]:>18.2f}| {commission:>18.2f}|")
    print('-'*60)
    print()
except Exception as e:
    print(f"Error calculating commissions: {e}")

```

1. โหลดข้อมูลการขาย

- เริ่มต้นโดยเรียกใช้ load_data เพื่อโหลดข้อมูลการขายจากไฟล์ sales.txt และข้อมูลพนักงานจากไฟล์ staff.txt
- ข้อมูลพนักงาน (staff) จะถูกจัดเก็บใน staff_dict โดยใช้ staff_id เป็น key และชื่อพนักงาน (name) เป็น value เพื่อให้เรียกใช้งานง่ายขึ้นเมื่อสร้างรายงานในขั้นตอนถัดไป

2. การคำนวณยอดขายรวมของพนักงานแต่ละคน

- ฟังก์ชันจะวนลูปผ่านรายการ sales ที่มีข้อมูลการขายแต่ละรายการ
- ดึง staff_id และยอดขาย (total) ของรายการนั้น แล้วแปลง total ให้เป็น float
- ใช้ sales_by_staff เพื่อเก็บยอดขายรวมของพนักงานแต่ละคน โดย
 - ถ้าพนักงานคนนั้นมีรายการการขายใน sales_by_staff แล้ว ก็จะเพิ่มยอดขายในรายการนั้น
 - ถ้ายังไม่มีรายการการขายของพนักงานคนดังกล่าวใน sales_by_staff ก็จะสร้างรายการใหม่โดยกำหนดยอดขายเป็นค่าเริ่มต้น

3. การคำนวณค่าคอมมิชชั่นของพนักงาน

- ฟังก์ชันจะวนลูปผ่าน sales_by_staff เพื่อคำนวณค่าคอมมิชชั่นสำหรับพนักงานแต่ละคน
- เรียกใช้ calculate_commission_rate โดยส่ง total_sales (ยอดขายรวมของพนักงานคนนั้น) เข้าไปเพื่อหาค่า commission_rate
- คำนวณค่าคอมมิชชั่นโดยการคูณ total_sales กับ commission_rate และเก็บผลลัพธ์ไว้ใน commissions โดยใช้ staff_id เป็น key

4. แสดงผลค่าคอมมิชชั่นพนักงานในรูปแบบตาราง

- ฟังก์ชันจะแสดงรายงานค่าคอมมิชชั่นในรูปแบบตารางโดยแบ่งออกเป็นคอลัมน์ Staff Name (ชื่อพนักงาน), Total Sales (Baht) (ยอดขายรวม), และ Commission (Baht) (ค่าคอมมิชชั่น)
- ใช้ staff_dict เพื่อค้นหาชื่อพนักงานจาก staff_id ใน commissions ถ้าพนักงานคนนั้นไม่พบใน staff_dict จะใช้ "Unknown Staff" แทน
- แต่ละแถวจะแสดงข้อมูลของพนักงานตามลำดับ พร้อมทั้งจัดตำแหน่งข้อมูลในคอลัมน์ให้เรียบร้อย

หากเกิดข้อผิดพลาดในขั้นตอนใดๆ ฟังก์ชันจะจับข้อผิดพลาดและแสดงข้อความ “Error calculating commissions” พร้อมรายละเอียดของข้อผิดพลาด (e)

3.9 โปรแกรมหลัก

โปรแกรมหลักที่สร้างขึ้นนี้เป็นโปรแกรมสำหรับการจัดการร้านกาแฟ โดยทำหน้าที่จัดการเมนู รายการขาย พนักงาน และค่าคอมมิชชั่น โดยใช้เมนูให้ผู้ใช้สามารถเลือกคำสั่งต่างๆ ได้ มีการทำงานหลักดังนี้

1. ฟังก์ชัน display_menu

```
def display_menu():
    print('='*34)
    print('Coffee Shop'.center(34))
    print('='*34)
    print("1. Add menu item (drink or bakery)")
    print("2. Add staff")
    print("3. Record sale")
    print("4. Cancel Order")
    print("5. Daily sales report")
    print("6. Sales report by staff")
    print("7. Calculate staff commissions")
    print("0. Exit")
    print('='*34)
```

- ฟังก์ชันนี้ใช้แสดงเมนูหลักของโปรแกรมเพื่อให้ผู้ใช้เห็นตัวเลือกต่าง ๆ ที่สามารถทำได้
- ใช้คำสั่ง print เพื่อสร้างเส้นและจัดรูปแบบให้สวยงาม มี 8 ตัวเลือกให้เลือก
 1. เพิ่มเมนู (เครื่องดื่มหรือเบเกอรี่)
 2. เพิ่มพนักงาน
 3. บันทึกการขาย
 4. ยกเลิกออเดอร์
 5. รายงานยอดขายประจำวัน
 6. รายงานยอดขายแยกตามพนักงาน
 7. คำนวณค่าคอมมิชชั่นพนักงาน
 0. ออกจากโปรแกรม

2. ฟังก์ชัน main

ฟังก์ชันนี้เป็นฟังก์ชันหลักของโปรแกรม ทำงานอยู่ใน while ลูป โดยจะวนแสดงเมนูและรอให้ผู้ใช้เลือกคำสั่งจนกว่าจะกด "0" เพื่อออกจากโปรแกรม มีขั้นตอนดังนี้

```
def main():
    while True:
        display_menu()
        choice = input("Select an option: ")
        print()
        try:
            if choice == '1':
                cs.add_menu()

            elif choice == '2':
                cs.add_staff()

            elif choice == '3':
                cs.add_sale()

            elif choice == '4':
                cs.cancel_order()

            elif choice == '5':
                cs.generate_daily_sales_report()

            elif choice == '6':
                cs.generate_sales_report_by_staff()

            elif choice == '7':
                cs.calculate_commissions()

            elif choice == '0':
                print("Exit Program")
                break

        else:
            print("Invalid option, please try again.")
```

- ขั้นตอนการทำงานของเงื่อนไขแต่ละคำสั่ง

เลือก '1' เรียกใช้ cs.add_menu() เพื่อเพิ่มเมนูใหม่ (เครื่องดื่มหรือเบเกอรี่)

เลือก '2' เรียกใช้ cs.add_staff() เพื่อเพิ่มหรือจัดการพนักงานในระบบ

เลือก '3' เรียกใช้ cs.add_sale() เพื่อบันทึกการขายรายการต่าง ๆ

เลือก '4' เรียกใช้ cs.cancel_order() เพื่อยกเลิกออเดอร์ที่บันทึกไว้

เลือก '5' เรียกใช้ cs.generate_daily_sales_report() เพื่อสร้างรายงานสรุปยอดขาย

ประจำวัน

เลือก '6' เรียกใช้ `cs.generate_sales_report_by_staff()` เพื่อสร้างรายงานยอดขายแยกตามพนักงาน

เลือก '7' เรียกใช้ `cs.calculate_commissions()` เพื่อคำนวณค่าคอมมิชชั่นของพนักงานตามยอดขายรวม

เลือก '0' พิมพ์ "Exit Program" และออกจากลูป `while` เพื่อสิ้นสุดโปรแกรม

```
except ValueError as ve:
    print(f"Invalid input: {ve}")
except FileNotFoundError as fnfe:
    print(f"File not found: {fnfe}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

จัดการข้อผิดพลาด โดยใช้ `try-except` เพื่อให้โปรแกรมไม่หยุดทำงานทันทีเมื่อเกิดข้อผิดพลาด

- `ValueError`: จับข้อผิดพลาดที่เกี่ยวข้องกับข้อมูลที่ผู้ใช้ป้อนผิด เช่น พิมพ์ตัวอักษรแทนตัวเลข
- `FileNotFoundError`: จับข้อผิดพลาดกรณีไม่พบไฟล์ที่ใช้บันทึกหรือโหลดข้อมูล เช่น ไฟล์ `sales.txt` หรือ `staff.txt`
- `Exception`: จับข้อผิดพลาดทั่วไปอื่น ๆ ที่อาจเกิดขึ้น

3. การทำงานทั้งหมด

- โปรแกรมจะวนลูปแสดงเมนูหลักให้ผู้ใช้เลือกคำสั่ง
- เมื่อเลือกคำสั่งใด ๆ โปรแกรมจะเรียกฟังก์ชันในโมดูล `coffee_shop` ที่เกี่ยวข้องเพื่อทำงานตามที่เลือก
- หากเกิดข้อผิดพลาด โปรแกรมจะจับข้อผิดพลาดและแสดงข้อความที่เหมาะสม
- ผู้ใช้สามารถเลือก "0" เพื่อสิ้นสุดโปรแกรม

ผลการดำเนินงาน

จากการศึกษาการออกแบบและพัฒนาโปรแกรมระบบร้านกาแฟด้วยภาษาไพทอน ครั้งนี้ ผู้จัดทำ สรุปผลการศึกษา ออกแบบระบบพัฒนาโปรแกรม การนำเทคโนโลยีมาช่วย เพื่อความสะดวกในการทำงานให้ง่ายขึ้น ช่วยลดระยะเวลาในการคำนวณยอดขายในปัจจุบัน และการคำนวณ Commission ช่วยในการจัดเก็บข้อมูลของพนักงาน

หน้าเมนูคำสั่งต่างๆ ผู้ใช้สามารถเลือกคำสั่งต่างๆได้จากหน้าจอแสดงเมนู โดยพิมพ์ตัวเลขลงไป

```
=====
                        Coffee Shop
=====
1. Add menu item (drink or bakery)
2. Add staff
3. Record sale
4. Cancel Order
5. Daily sales report
6. Sales report by staff
7. Calculate staff commissions
0. Exit
=====
Select an option: █
```

ภาพที่ 4 หน้าเมนู

หน้าจอแสดงตัวเลือกที่ 1 เมื่อต้องการเพิ่มเครื่องดื่ม หรือเบเกอรี่ ผู้ใช้สามารถเพิ่มรหัส ชื่อ และราคาของเมนูได้ เมื่อใช้คำสั่ง add

```
Select an option: 1

Do you want to add, edit, or delete an item? (add/edit/delete): add
Enter item ID: 101
Enter item name: Milk Tea
Enter price: 40
Milk Tea has been added to the menu.

-----
                        Current Menu
-----
|Item ID|  Menu Item  | Price (Baht) |
-----
|101    | Milk Tea    |         40.00|
-----
```

ภาพที่ 5 หน้าจอแสดงการเพิ่มเมนู

ถ้าผู้ใช้อยากแก้ไขชื่อและราคาของเมนู สามารถใช้คำสั่ง edit ได้

```
Add, edit, or delete another item? (y/n): y
Do you want to add, edit, or delete an item? (add/edit/delete): edit
Enter item ID to edit: 102
Editing Item ID 102: Cheese Cake - Current Price: 60.0
Enter new name for Cheese Cake (leave blank to keep current name): Brownie
Enter new price for Cheese Cake (current price: 60.0): 35
Item ID 102 has been updated.
-----
Current Menu
-----
|Item ID| Menu Item | Price (Baht) |
-----
|101| Milk Tea | 40.00|
|102| Brownie | 35.00|
-----
```

ภาพที่ 6 หน้าจอแสดงการแก้ไขเมนู

ถ้าผู้ใช้อยากลบเมนูในรายการ สามารถใช้คำสั่ง delete ได้

```
Add, edit, or delete another item? (y/n): y
Do you want to add, edit, or delete an item? (add/edit/delete): delete
Enter item ID to delete: 102
Item ID 102 has been deleted from the menu.
-----
Current Menu
-----
|Item ID| Menu Item | Price (Baht) |
-----
|101| Milk Tea | 40.00|
-----
```

ภาพที่ 7 หน้าจอแสดงการลบข้อมูลของเมนู

หน้าจอแสดงตัวเลือกที่ 2 เมื่อต้องการเพิ่มพนักงานขาย ผู้ใช้สามารถเพิ่มรหัส และชื่อของพนักงานขายได้ เมื่อใช้คำสั่ง add

```
Select an option: 2

Do you want to add, edit, or delete a staff member? (add/edit/delete): add
Enter staff ID: A001
Enter staff name: Jane
Jane has been added to the staff list.
-----
Current Staff
-----
|Staff ID | Staff Name |
-----
|A001 | Jane |
-----
```

ภาพที่ 8 หน้าจอแสดงการเพิ่มพนักงานขาย

ถ้าผู้ใช้อยากแก้ไขชื่อของพนักงานชาย สามารถใช้คำสั่ง edit ได้

```
Add, edit, or delete another staff member? (y/n): y
Do you want to add, edit, or delete a staff member? (add/edit/delete): edit
Enter staff ID to edit: A002
Editing Staff ID A002: Tony
Enter new name for Tony (leave blank to keep current name): Peter
Staff ID A002 has been updated.
-----
Current Staff
-----
|Staff ID   |Staff Name |
-----
|A001      |Jane       |
|A002      |Peter      |
-----
```

ภาพที่ 9 หน้าจอแสดงการแก้ไขข้อมูลของพนักงานชาย

ถ้าผู้ใช้อยากลบข้อมูลของพนักงานชาย สามารถใช้คำสั่ง delete ได้

```
Add, edit, or delete another staff member? (y/n): y
Do you want to add, edit, or delete a staff member? (add/edit/delete): delete
Enter staff ID to delete: A002
Staff ID A002 has been deleted from the staff list.
-----
Current Staff
-----
|Staff ID   |Staff Name |
-----
|A001      |Jane       |
-----
```

ภาพที่ 10 หน้าจอแสดงการลบข้อมูลของพนักงานชาย

หน้าจอแสดงตัวเลือกที่ 3 เมื่อต้องการบันทึกการขาย โดยใส่รหัสพนักงานขาย รหัสเมนู และจำนวน แล้วจะแสดงออกมาในรูปแบบตาราง

```
Select an option: 3

Enter staff ID: A001
Enter item ID: 101
Enter quantity: 3
Do you want to add another item? (y/n): n
=====
                        Receipt
=====
|Item           | Qty|   Price|   Total|
-----
|Milk Tea       |   3|   40.00|  120.00|
-----
|Total          |    |        | 120.00 Baht|
=====
Sales recorded by staff: Jane
Thank you for your purchase!
```

ภาพที่ 11 หน้าจอแสดงการบันทึกการขาย

ในกรณีที่ใส่สองเมนู จะใส่แค่รหัสเมนู และจำนวน

```
Enter staff ID: A001
Enter item ID: 101
Enter quantity: 2
Do you want to add another item? (y/n): y
Enter item ID: 102
Enter quantity: 1
Do you want to add another item? (y/n): n
=====
                        Receipt
=====
|Item           | Qty|   Price|   Total|
-----
|Milk Tea       |   2|   40.00|   80.00|
|CupCake        |   1|   35.00|   35.00|
-----
|Total          |    |        | 115.00 Baht|
=====
Sales recorded by staff: Jane
Thank you for your purchase!
```

ภาพที่ 12 หน้าจอแสดงการบันทึกการขาย

หน้าจอแสดงตัวเลือกที่ 4 เมื่อต้องการยกเลิกออเดอร์ โดยสามารถเลือกหมายเลขออเดอร์ที่ต้องการยกเลิกได้

```
Select an option: 4

=====
Current Orders
=====
|Order No.|Item ID |Staff ID | Qty |Total (Baht)|
-----
|1         |101     |A001     | 3   |120.00|
|2         |101     |A001     | 2   |80.00|
|3         |102     |A001     | 1   |35.00|
-----

Enter the Order No. to cancel: 1
Order for Item ID 101 by Staff ID A001 has been canceled.
```

ภาพที่ 13 หน้าจอแสดงการยกเลิกออเดอร์

หน้าจอแสดงตัวเลือกที่ 5 เมื่อต้องการออกรายงานสรุปรายยอดขายประจำวัน

```
Select an option: 5

-----
Daily Sales Report
-----
|Menu Item   | Quantity Sold | Total (Baht)|
-----
|Milk Tea    | 2             | 80.00|
|CupCake     | 1             | 35.00|
-----
|Total Sales |               | 115.00|
-----
```

ภาพที่ 14 หน้าจอแสดงการออกรายงานสรุปรายยอดขายประจำวัน

หน้าจอแสดงตัวเลือกที่ 6 เมื่อต้องการออกรายงานสรุปรายยอดขายแยกตามพนักงาน

```
Select an option: 6

-----
Sales Report by Staff
-----
| Staff Name | Total (Baht) |
-----
|Jane        | 115.00|
-----
```

ภาพที่ 15 หน้าจอแสดงการออกรายงานสรุปรายยอดขายแยกตามพนักงาน

หน้าจอแสดงตัวเลือกที่ 7 เมื่อต้องการคิดค่าคอมมิชชั่นของพนักงาน

Select an option: 7

Staff Commissions		
Staff Name	Total Sales (Baht)	Commission (Baht)
Jane	115.00	2.30

ภาพที่ 16 หน้าจอแสดงการคิดค่าคอมมิชชั่นของพนักงาน

หน้าจอแสดงตัวเลือกที่ 0 เมื่อต้องการออกจากโปรแกรม

```

=====
                        Coffee Shop
=====
1. Add menu item (drink or bakery)
2. Add staff
3. Record sale
4. Cancel Order
5. Daily sales report
6. Sales report by staff
7. Calculate staff commissions
0. Exit
=====
Select an option: 0

Exit Program
  
```

ภาพที่ 17 หน้าจอแสดงออกจากโปรแกรม

สรุปผลการดำเนินงาน

โปรแกรมระบบร้านกาแฟ เป็นโปรแกรมที่สามารถรองรับการทำงานด้านการจัดการร้านกาแฟได้อย่างครบถ้วน ตั้งแต่การเพิ่มเมนูและพนักงาน บันทึกยอดขาย สร้างรายงาน ไปจนถึงการคำนวณค่าคอมมิชชั่นพนักงาน จึงเป็นเครื่องมือที่มีประโยชน์สำหรับผู้จัดการร้านหรือเจ้าของร้านในการบริหารจัดการร้านกาแฟให้มีประสิทธิภาพ และผู้ที่สนใจสามารถนำโปรแกรมนี้อไปใช้ต่อยอดหรือนำไปศึกษาต่อได้ในอนาคต

ข้อเสนอแนะ

1. โปรแกรมควรมีการเพิ่มระบบเข้าทำงานของพนักงานตามช่วงเวลาต่างๆ
2. โปรแกรมควรมีระบบเงินเดือนของพนักงาน

บรรณานุกรม